

Spherical Harmonics Lighting System

Developer & AI-Agent Reference —v2026

0. Purpose and How to Use This Document

This document is the primary technical reference for the Spherical Harmonics (SH) lighting toolkit for TouchDesigner. It is written to serve two audiences equally:

1. Human developers who want to extend the system or write custom shaders.
2. AI coding agents / LLMs that will be given this PDF alongside the TouchDesigner documentation and asked to implement custom generators, materials, or particle behaviours on behalf of a user who may not know GLSL.

All code blocks in this document are production-ready and follow the exact naming conventions used in the supplied shader files. An AI agent can copy any block verbatim and expect it to work in a GLSL POP Compute or GLSL MAT Pixel shader.

AI AGENT STARTING POINT: Read sections 1–4 to understand data layout and coefficient ordering. Then read the section that matches your task: section 5 for a new generator, section 6 for a new material, section 7 for particle behaviours. Section 8 gives complete copy-paste function blocks.

1. System Overview

The system stores low-frequency diffuse light as Spherical Harmonics (SH) coefficients in a 3D grid of "probes" that covers the scene in world space. Light data travels through four stages:

Stage 1 —Generate/Calculate: A GLSL POP Compute shader writes SH coefficients onto probe points (POP attributes).

Stage 2 —Modify: Optional GLSL POP Compute shaders reshape the field (band filter, sigma windowing, HSV, rotation, etc.).

Stage 3 —Transfer: A GLSL TOP Compute shader (POP2T3D) reads POP attributes and writes them into nine 3D textures, one texel per probe.

Stage 4 —Render: GLSL MAT pixel shaders sample the 3D textures at each fragment's world-space position and reconstruct diffuse irradiance from the stored coefficients and the surface normal.

The system handles DIFFUSE lighting only. Specular, glossy reflections, and anisotropy are outside its scope. The current version does not compute occlusion between light sources and probes.

2. SH Coefficient Layout —TheCore Contract

L2 Spherical Harmonics give 9 basis functions per colour channel. With R, G, B that is 27 scalar values per probe. These 27 values are grouped into 9 vec4 POP attributes (one per basis-function group).

2a. POP Attribute Names and Packing

```
// Nine POP attributes per probe – this layout is the contract shared
// between ALL generators, calculators, modifiers, and the POP2T3D shader.
```

```
// RED channel
vec4 shR0 = [ Y00 , Y1-1 , Y10 , Y11 ] // L0 + L1
```

```

vec4 shR1 = [ Y2-2, Y2-1, Y20, Y21 ] // L2 (first 4)
vec4 shR2 = [ Y22, 0, 0, 0 ] // L2 (last 1, x component only)

// GREEN channel – identical structure
vec4 shG0, shG1, shG2

// BLUE channel – identical structure
vec4 shB0, shB1, shB2

```

shR2/shG2/shB2 are vec4 in the texture representation but only the .x component (Y22) carries data. The .yzw are always zero. In some older POP shaders these were stored as a float attribute —always treat them as vec4 when writing to 3D textures.

2b. SH Basis Constants (must match in EVERY shader)

```

// Copy these constants verbatim into any shader that reads or writes SH data.
// Any mismatch between generator and decoder produces wrong lighting.

const float SH_C0 = 0.28209479177387814; // 1 / (2*sqrt(pi))
const float SH_C1 = 0.48860251190291992; // sqrt(3) / (2*sqrt(pi))
const float SH_C2_1 = 1.09254843059207907; // sqrt(15) / (4*pi)
const float SH_C2_2 = 0.31539156525252005; // sqrt(5) / (16*pi)
const float SH_C2_3 = 0.54627421529603954; // sqrt(15) / (16*pi)

// Cosine-lobe (Lambertian) convolution weights – bake into generators/calculators,
// NOT into material shaders (which do direct dot-product reconstruction).
const float A0 = 3.14159265358979323846; // pi (band 0)
const float A1 = 2.09439510239319549231; // 2*pi/3 (band 1)
const float A2 = 0.78539816339744830962; // pi/4 (band 2)

```

The cosine-lobe weights A0, A1, A2 are baked into the output by calculators/generators during accumulation. This is why material shaders can perform a simple dot product to recover irradiance —no convolution is needed at render time.

3. 3D Texture Layout (Stage 3 —POPto Textures)

The POP2T3D compute shader converts POP attributes into nine RGBA 3D textures. Each texel corresponds to exactly one probe. The 3D texture resolution must match the probe grid: width=X, height=Y, depth=Z.

Texture names and contents

Texture	Contents (vec4)	Notes
SH_R0	[Y00, Y1-1, Y10, Y11] – red L0+L1	sampler3D in MAT
SH_R1	[Y2-2, Y2-1, Y20, Y21] – red L2a	sampler3D in MAT
SH_R2	[Y22, 0, 0, 0] – red L2b	x component only
SH_G0	same layout, green channel	
SH_G1	same layout, green channel	
SH_G2	same layout, green channel	
SH_B0	same layout, blue channel	
SH_B1	same layout, blue channel	
SH_B2	same layout, blue channel	

Why 9 RGB textures instead of 7 RGBA? One texture maps to one POP attribute group. This makes the relationship between POP data and texture data explicit and easy to debug. The three unused channels in SH_R2/G2/B2 are a small, acceptable cost.

Probe index formula (used in POP2T3D and in any generator writing in grid order)

```

// Linear probe index from 3D grid coordinates.
// This is the ordering assumed by ALL parts of the pipeline.
uint probeIndex = uint(coord.x
    + coord.y * dim.x

```

```
+ coord.z * dim.x * dim.y));
```

4. Sampling the Probe Grid in a Material Shader

4a. Required uniforms and samplers (declare in every grid-based MAT pixel shader)

```
// ---- SH probe grid uniforms ----
uniform vec3  uGridOrigin;           // world-space centre of the probe grid
uniform vec3  uGridSize;             // world-space extents (width, height, depth)
uniform float uSHHighlightRolloff;   // Reinhard white-point; start at 2.0, tune visually

// ---- SH probe grid 3D texture samplers ----
// In GLSL MAT, sampler uniforms must be declared explicitly.
uniform sampler3D SH_R0;
uniform sampler3D SH_R1;
uniform sampler3D SH_R2;
uniform sampler3D SH_G0;
uniform sampler3D SH_G1;
uniform sampler3D SH_G2;
uniform sampler3D SH_B0;
uniform sampler3D SH_B1;
uniform sampler3D SH_B2;
```

4b. fetchGridSH —world-spacelookup (copy verbatim above void main())

```
// Samples all 9 SH coefficient textures at a world-space position.
// Grid dimensions are derived from textureSize at runtime – no resolution uniform needed.
void fetchGridSH(vec3 worldPos,
    out vec4 shR0, out vec4 shR1, out vec4 shR2,
    out vec4 shG0, out vec4 shG1, out vec4 shG2,
    out vec4 shB0, out vec4 shB1, out vec4 shB2)
{
    vec3 gridMin = uGridOrigin - uGridSize * 0.5;
    vec3 uvw01   = (worldPos - gridMin) / uGridSize;

    // Remap to texel centres – prevents bleeding at grid edges.
    ivec3 dimI   = textureSize(SH_R0, 0);
    vec3  dim    = vec3(dimI);
    vec3  invDim = 1.0 / dim;
    vec3  uvw    = uvw01 * ((dim - 1.0) * invDim) + 0.5 * invDim;
    uvw = clamp(uvw, 0.5 * invDim, 1.0 - 0.5 * invDim);

    shR0 = texture(SH_R0, uvw);  shR1 = texture(SH_R1, uvw);  shR2 = texture(SH_R2, uvw);
    shG0 = texture(SH_G0, uvw);  shG1 = texture(SH_G1, uvw);  shG2 = texture(SH_G2, uvw);
    shB0 = texture(SH_B0, uvw);  shB1 = texture(SH_B1, uvw);  shB2 = texture(SH_B2, uvw);
}
```

4c. evalSHL2Irradiance —decodeirradiance from coefficients and a normal (copy verbatim)

```
// Evaluates L2 SH irradiance for a normalised world-space surface normal n.
// The stored coefficients already include cosine-lobe weighting (A0/A1/A2),
// so this is a direct dot product – no extra convolution needed.
// Returns RGB irradiance (can be negative near dark edges; clamp after calling).
vec3 evalSHL2Irradiance(vec3 n,
    vec4 shR0, vec4 shR1, vec4 shR2,
    vec4 shG0, vec4 shG1, vec4 shG2,
    vec4 shB0, vec4 shB1, vec4 shB2)
{
    float x = n.x, y = n.y, z = n.z;

    vec4 y0 = vec4(SH_C0,
        SH_C1 * y,
        SH_C1 * z,
        SH_C1 * x);
```

```

vec4 y1 = vec4(SH_C2_1 * x * y,
               SH_C2_1 * y * z,
               SH_C2_2 * (3.0 * z * z - 1.0),
               SH_C2_1 * x * z);
float y2 = SH_C2_3 * (x * x - y * y);

return vec3(
    dot(shR0, y0) + dot(shR1, y1) + shR2.x * y2,
    dot(shG0, y0) + dot(shG1, y1) + shG2.x * y2,
    dot(shB0, y0) + dot(shB1, y1) + shB2.x * y2
);
}

```

4d. Highlight compression block (insert inside void main() after lighting loops)

```

// Fetch SH coefficients at this fragment's world position.
vec4 shR0, shR1, shR2, shG0, shG1, shG2, shB0, shB1, shB2;
fetchGridSH(iVert.worldSpacePos.xyz,
    shR0, shR1, shR2,
    shG0, shG1, shG2,
    shB0, shB1, shB2);

// Evaluate irradiance; clamp negative lobes.
vec3 shDiffuse = evalSHL2Irradiance(normal,
    shR0, shR1, shR2,
    shG0, shG1, shG2,
    shB0, shB1, shB2);
shDiffuse = max(shDiffuse, vec3(0.0));

// Extended Reinhard highlight compression – applied to SH term only.
// uSHHighlightRolloff is a "white point": values well below it pass
// nearly unchanged; values above it roll off smoothly.
// Higher value = softer rolloff; lower = more compression.
// Default 2.0. Useful range 1.0 – 8.0.
float shW2 = uSHHighlightRolloff * uSHHighlightRolloff;
shDiffuse = shDiffuse * (1.0 + shDiffuse / shW2) / (1.0 + shDiffuse);

// For Phong / normal-mapped Phong:
lightingColor += shDiffuse;

// For PBR (respects metallic mask – fully metallic = no SH diffuse):
// lightingColor += shDiffuse * pbrDiffuseColor;

```

Always pass the same "normal" variable that was used for TDLighting / TDEnvLightingPBR. For normal-mapped variants this is the post-TBN-transform perturbed normal. For basic Phong it is the world-space normal after the TDFrontFacing flip.

5. Writing a Custom Generator (GLSL POP Compute)

A generator is a GLSL POP Compute shader that writes SH coefficients onto a probe grid. It receives one POP input (the probe grid with P attribute) and writes shR0..shB2 attributes on each probe.

5a. The key accumulation function —shL2IrradianceBasis

```

// Evaluates the L2 SH basis for a direction and bakes in cosine-lobe weights.
// The result is ready to be multiplied by a light colour and accumulated.
void shL2IrradianceBasis(vec3 dir,
    out vec4 b0, out vec4 b1, out float b2)
{
    float x = dir.x, y = dir.y, z = dir.z;
    b0 = vec4(SH_C0 * A0,
              SH_C1 * y * A1,
              SH_C1 * z * A1,
              SH_C1 * x * A1);
    b1 = vec4(SH_C2_1 * x*y * A2,

```

```

        SH_C2_1 * y*z          * A2,
        SH_C2_2 * (3.0*z*z-1.0) * A2,
        SH_C2_1 * x*z          * A2);
    b2 = SH_C2_3 * (x*x - y*y) * A2;
}

// Usage - accumulate one directional light source:
void accumulateLight(vec3 lightDir, vec3 lightColor,
    inout vec4 shRAcc0, inout vec4 shRAcc1, inout float shRAcc2,
    inout vec4 shGAcc0, inout vec4 shGAcc1, inout float shGAcc2,
    inout vec4 shBAcc0, inout vec4 shBAcc1, inout float shBAcc2)
{
    vec4 b0; vec4 b1; float b2;
    shL2IrradianceBasis(normalize(lightDir), b0, b1, b2);
    shRAcc0 += lightColor.r * b0; shRAcc1 += lightColor.r * b1; shRAcc2 += lightColor.r * b2;
    shGAcc0 += lightColor.g * b0; shGAcc1 += lightColor.g * b1; shGAcc2 += lightColor.g * b2;
    shBAcc0 += lightColor.b * b0; shBAcc1 += lightColor.b * b1; shBAcc2 += lightColor.b * b2;
}

```

5b. Generator main() skeleton

```

void main()
{
    uint probeId = TDIndex();
    if (probeId >= TDNumElements()) return;

    vec3 probePos = TDIn_P(0, probeId); // world-space probe position

    // Initialise accumulators.
    vec4 shRAcc0 = vec4(0.0), shRAcc1 = vec4(0.0); float shRAcc2 = 0.0;
    vec4 shGAcc0 = vec4(0.0), shGAcc1 = vec4(0.0); float shGAcc2 = 0.0;
    vec4 shBAcc0 = vec4(0.0), shBAcc1 = vec4(0.0); float shBAcc2 = 0.0;

    // --- YOUR LIGHT ACCUMULATION LOGIC HERE ---
    // Example: one directional light from above
    vec3 lightDir = normalize(vec3(0.3, 1.0, 0.2));
    vec3 lightColor = vec3(1.0, 0.9, 0.7) * 2.0;
    accumulateLight(lightDir, lightColor,
        shRAcc0, shRAcc1, shRAcc2,
        shGAcc0, shGAcc1, shGAcc2,
        shBAcc0, shBAcc1, shBAcc2);
    // -----

    // Write output attributes.
    shR0[probeId] = shRAcc0; shR1[probeId] = shRAcc1;
    shR2[probeId] = vec4(shRAcc2, 0.0, 0.0, 0.0);
    shG0[probeId] = shGAcc0; shG1[probeId] = shGAcc1;
    shG2[probeId] = vec4(shGAcc2, 0.0, 0.0, 0.0);
    shB0[probeId] = shBAcc0; shB1[probeId] = shBAcc1;
    shB2[probeId] = vec4(shBAcc2, 0.0, 0.0, 0.0);
}

```

5c. Point / sphere light falloff pattern (from the supplied calculator)

```

// Physically-correct inverse-square falloff, bounded at uFalloffRadius.
// At d = uFalloffRadius → atten = 0.5. At d = 4*uFalloffRadius → atten ≈ 0.06.
uniform float uFalloffRadius;
float pointFalloff(float dist2)
{
    float r = max(uFalloffRadius, 1e-6);
    return 1.0 / (1.0 + dist2 / (r * r));
}

// Safe normalize (avoids NaN when length → 0)
vec3 safeNorm(vec3 v)
{

```

}

li

```
// TouchDesigner auto-generates TDI * ***** fun
```

```
// TouchDesigner auto-generates TDIIn_* accessor functions for every attribute
```

T

```
// ADD NEAR THE TOP (1st position)
```

[illegible]

```
// In the GLSL MAT operator UT:
```

//

```
// Vectors/Floats page:
```

```
// uGridOrigin      vec3    world-space XYZ centre of probe grid
// uGridSize        vec3    world-space XYZ extents of probe grid
// uSHHighlightRolloff float  start at 2.0, adjust visually
//
// Samplers page (type = sampler3D for all nine):
// SH_R0 → 3D texture TOP (red L0 band)
// SH_R1 → 3D texture TOP (red L1 band) ... etc. for all 9
//
// The 3D textures come from the POP2T3D conversion component.
// Their resolution must match the probe grid resolution.
```

6c. Adding a Color Map (example of a common customisation)

```
// To add a colour map texture to an SH material:

// 1. Add these declarations near the top:
uniform sampler2D sColorMap;

// 2. In void main(), sample it using UV coordinates:
vec4 colorMapSample = texture(sColorMap, iVert.texCoord0_Tex.st);

// 3. Use it to modulate the SH diffuse contribution:
lightingColor += shDiffuse * colorMapSample.rgb;

// Or multiply the whole final colour:
finalColor.rgb *= colorMapSample.rgb;
```

7. Using SH Data for Particle Behaviours (GLSL POP Compute)

Because SH coefficients are stored as ordinary POP attributes, they are available to any GLSL POP Compute shader. This opens up a wide range of particle-behaviour possibilities: you can steer particles toward bright light, colour them by local light intensity, modulate forces, or drive any other attribute.

7a. Reading SH data from a probe grid in a GLSL POP Compute shader

If the probe grid is connected as input 1 (or another input), you can sample it by finding the nearest probe or by doing a trilinear lookup manually. The simplest approach is to use the Neighbor POP to find the nearest probe and read its coefficients directly.

```
// In a GLSL POP Compute shader operating on particles (input 0),
// with a probe grid connected as input 1:

uint probeId = TDIn_Nebr(0, particleId, 0); // nearest probe index
if (probeId == 4294967295u) return;

// Read the probe's SH coefficients.
vec4 pShR0 = TDIn_shR0(1, probeId); // L0+L1 red
vec4 pShG0 = TDIn_shG0(1, probeId);
vec4 pShB0 = TDIn_shB0(1, probeId);
// Y00 (the ambient / average light) is always in the .x component of sh?0.
float avgBrightness = (pShR0.x + pShG0.x + pShB0.x) / 3.0;
// Normalise: the physical expected value of Y00 (C0 * A0) is ~0.886
// so avgBrightness in physical units is around 1.0 for a fully-lit probe.
```

7b. Extracting dominant light direction from L1 coefficients

```
// The L1 coefficients (Y1-1, Y10, Y11) encode the main light gradient direction.
// Extracting them gives an approximate "direction to brightest light" per probe.
// This is a rough estimate (uses only L1 bands, not the full L2 field).

// From sh?0: layout is [Y00, Y1-1, Y10, Y11]
// Y1-1 → y axis, Y10 → z axis, Y11 → x axis

vec4 shR0 = TDIn_shR0(1, probeId);
vec4 shG0 = TDIn_shG0(1, probeId);
```

```

vec4 shB0 = TDIn_shB0(1, probeId);

// Average over colour channels to get a single luminance gradient.
vec3 L1avg = (shR0.yzw + shG0.yzw + shB0.yzw) / 3.0;

// L1avg.x = Y1-1 (y axis component)
// L1avg.y = Y10 (z axis component)
// L1avg.z = Y11 (x axis component)
// So dominant direction in world space (xyz) = (L1avg.z, L1avg.x, L1avg.y)

vec3 dominantDir = vec3(L1avg.z, L1avg.x, L1avg.y);
float magnitude = length(dominantDir);
if (magnitude > 1e-6)
    dominantDir = dominantDir / magnitude;

// Use dominantDir to steer particles:
vec3 particleVel = TDIn_v(0, particleId);
float steerStr = 0.5;
particleVel += dominantDir * steerStr;
v[particleId] = particleVel;

```

7c. Direct per-particle SH path (L1 only —fordense particle lighting)

For cases where you want each particle to be shaded by its own calculated SH field (without a probe grid), the project includes a separate L1 calculator. It stores only 3 float4 attributes per particle: shR, shG, shB.

```

// Coefficient layout for the direct L1 particle path:
// shR / shG / shB = vec4 [ Y00, Y11(x), Y1-1(y), Y10(z) ]
//
// Evaluating irradiance for a normal in the pixel shader:
const float SH_C0 = 0.28209479177387814;
const float SH_C1 = 0.48860251190291992;

vec3 evalSHL1Irradiance(vec3 n, vec4 shR, vec4 shG, vec4 shB)
{
    vec4 y = vec4(SH_C0,
                  SH_C1 * n.x,
                  SH_C1 * n.y,
                  SH_C1 * n.z);
    return vec3(dot(shR, y), dot(shG, y), dot(shB, y));
}

// In the pixel shader, fetch by instance ID:
uint uid = uint(iVert.instanceID);
vec4 shR = TDBuffer_shR(uid);
vec4 shG = TDBuffer_shG(uid);
vec4 shB = TDBuffer_shB(uid);
vec3 shDiffuse = evalSHL1Irradiance(normal, shR, shG, shB) * uSHIntensity;
lightingColor += max(shDiffuse, vec3(0.0));

```

8. Modifier Reference

Band Filter / Sigma Windowing (supplied as GLSL POP Compute)

```

// Sigma windowing – low-pass filter in SH band space.
// Reduces ringing (negative SH lobes) from sharp or high-contrast sources.
// Formula: w_l = exp(-lambda * l * (l+1))
// l=0: w0 = 1.0 (ambient – unchanged)
// l=1: w1 = exp(-2*λ) (directional gradient – slightly damped)
// l=2: w2 = exp(-6*λ) (fine directional – most damped)
// λ = 0 → no-op. λ = 0.2 → mild. λ = 1.0 → nearly collapses to ambient.
// Reference: P.-P. Sloan, "Stupid Spherical Harmonics (SH) Tricks", 2008.

uniform float uSigmaLambda; // range 0.0 .. 1.0

```



```

vec3 applySigmaToScale(vec3 s, float lambda)
{
    if (lambda == 0.0) return s;
    return vec3(s.x,
                s.y * exp(-2.0 * lambda),
                s.z * exp(-6.0 * lambda));
}

```

Per-band gains (uScaleL0/L1/L2, each a vec3 for R/G/B) and band tilt (uBandTilt, exponential, range -2 to +2) are also available in the supplied Band Filter component. These let you independently boost or attenuate ambient vs directional content per colour channel.

Combining multiple SH fields (field arithmetic)

```

// To combine (add) two SH fields that share the same probe grid,
// use the Blend POP in TouchDesigner with mode = Add.
// This is equivalent to adding coefficients element-wise and is mathematically correct.
//
// Math Mix POP can also do this, but requires writing formulas for all 9 attributes.
// Blend POP is strongly preferred for simple field combination.
//
// IMPORTANT: you can only combine fields that share IDENTICAL probe grids -
// same resolution, same world-space position, same spacing.
// The result of combining mismatched grids is undefined.

```

9. Known Limitations

These limitations are inherent to the current design and should be communicated to users:

- Diffuse only. The system cannot represent specular, glossy, or metallic reflections.
- No occlusion. Geometry between a light source and a probe does not block the contribution.
- Ringing. Very sharp or high-contrast light sources may produce false gradients between probes. Use sigma windowing (section 8) to reduce this.
- Grid coverage. Geometry outside the probe grid boundary is sampled at the nearest edge probe (clamped). No extrapolation occurs.

10. External References

Peter-Pike Sloan —"StupidSpherical Harmonics (SH) Tricks" (2008, GDC). The primary reference for real-time SH techniques. Covers basis functions, coefficient ordering, convolution, sigma windowing (used in the Band Filter modifier), and many practical shortcuts. Freely available online.

TouchDesigner GLSL MAT documentation —docs.derivative.ca. Essential for understanding TD-specific macros (TDLighting, TDDeform, TDPixelColor, etc.), the uniform auto-injection conventions, and how Buffers/Samplers pages work in the operator UI.

TouchDesigner GLSL POP documentation —docs.derivative.ca. Covers TDIndex(), TDNumElements(), TDIn_* attribute accessors, output attribute arrays, and the layout_local_size requirements for compute shaders.

The supplied shader files —these are the ground-truth implementation. In any conflict between this document and the actual shader code, the shader code wins.